# Early Warning of Model Degradation under Delayed Labels: A Pre-Registered Evaluation of Drift and Harm Signals

Akshay Naphade, Rushikesh Kadepurkar, Parth Tatpuje, Vishnu Gurav and Sunil Bhirud
*Department of Computer Engineering*, *COEP Technological University*, Pune, India
{[akshay-email], [rushikesh-email], [parth-email], [vishnu-email], [guide-email]}@[domain]

***Abstract*—Deployed classifiers often lose accuracy long before true labels reveal it. Drift detectors offer earlier signals, but most input shifts do not harm the model, and thresholds tuned on one dataset may not hold on another. We built a two-tier monitor and evaluated it with pre-registered hypotheses, frozen thresholds and one-time runs on unseen datasets. Streams re-sample real rows, so inputs drift from a known batch while every row keeps its true label. Our first pre-registered design, a learned risk calibrator over raw drift scores, failed all five hypotheses. Thresholds in raw units did not transfer: on unseen datasets, false alerts ranged from 1.2 to 44.6 per 100 clean batches against a budget of 5. Measuring each signal against the stream's own burn-in fixed this on benchmark streams. The resulting drift watch warned before the accuracy drop on 87% of harmful gradual shifts, a median of six batches early, at 4.4 false alerts per 100 clean batches. It did not work on natural streams. A second tier that estimates accuracy from an importance-weighted labeled reference cut alarms on harmless shifts from 97% to 34% and separated harmful from harmless shifts with an AUROC of 0.78 (95% CI 0.70–0.86), below our 0.80 target but above confidence-based baselines (0.56–0.59) on the same data. On six later datasets it exceeded the false-alert budget (12.7 per 100). Nonlinear estimators and ensembles did not beat it on development data, nor did stricter alerting rules on fresh data. We release the benchmark, code and all verdicts.**

***Index Terms*—concept drift, dataset shift, model monitoring, delayed labels, performance estimation, pre-registration, MLOps**

## I. Introduction

A classifier is trained on past data and deployed on a stream whose distribution changes. Whether the change hurts is known only once true labels arrive, which in credit, fraud or churn prediction can take days or weeks. Monitoring systems therefore watch label-free signals such as feature distributions and prediction confidence [1], [2]. They have two problems. Most distribution shifts do not hurt a given model [3], so drift alarms often fire for nothing. And alarm thresholds are tuned on historical data, but it is rarely checked whether a threshold still means the same thing on another dataset.

We want a monitor that warns when accuracy is about to fall, not whenever the inputs change, and that does so before delayed labels can confirm the fall. We froze every hypothesis, pass rule and alert threshold in code before downloading the data that would test it, and a script computed each verdict. This stopped us from choosing thresholds after seeing the results. The pre-registered work followed five exploratory studies (v1–v5) on 13 public datasets. The first pre-registered study (v6) reused those datasets; the later confirmations (v7, v8, v10) used datasets never used before.

Our contributions are:

1) A benchmark that re-samples real tabular data to create gradual, abrupt and irrelevant-feature shifts and label changes with a known onset, where whether a shift harms the model is measured from true labels.
2) Evidence that drift thresholds expressed in raw units do not transfer across datasets, and that expressing signals relative to a stream's own burn-in period largely does: an early drift warning that held its false-alert budget on unseen benchmark data, though not on natural streams.
3) A label-free harm tier based on importance-weighted reference accuracy. It cuts alarms on harmless shifts and separates harm better than confidence-based estimates, though it missed its own targets.
4) Pre-registered negative results: a failed first design, a failed second tier in v7, a benchmark gate that failed, and two attempts to exceed the harm-separation target (one failed its development gate, one failed on fresh data).

## II. Related Work

**Concept drift.** Surveys cover detection and adaptation under drift [4], [5]. Classical detectors such as DDM [6] and ADWIN [7] monitor the error stream and therefore need labels; sequential tests such as CUSUM [8] accumulate evidence of a persistent change. Label-free detection compares feature or embedding distributions with two-sample tests such as Kolmogorov–Smirnov or maximum mean discrepancy [9]; Rabanser et al. [3] evaluated such detectors and noted that detected shift need not reduce accuracy. Dataset shift and label shift are treated in [10], [11]. Podkopaev and Ramdas [12] also separate harmful from benign shifts. Their sequential tests use time-uniform confidence sequences to keep false alarms under control, but they need production labels, which may arrive late.

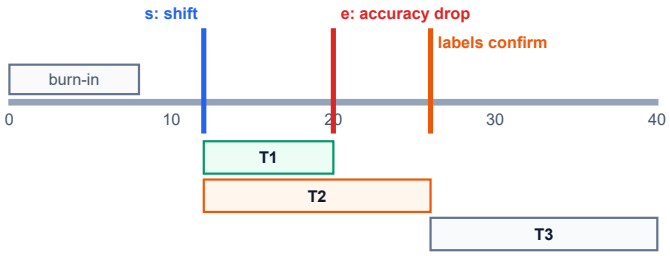



Fig. 1. Warning tiers for a shift at $s = 12$, an accuracy drop at $e = 20$ and label delay $L = 5$: T1 ends when accuracy drops, T2 when delayed labels could confirm it.

Our monitor acts before those labels arrive and has no such guarantee.

**Label-free performance estimation.** Confidence-based estimation (CBPE) [13], average thresholded confidence [14], difference of confidences [15] and model disagreement [16] estimate accuracy without labels. Under covariate shift, importance weighting reweights source samples by the density ratio [17], [18]. PAPE [19] takes such weights from a domain classifier and uses them to recalibrate the model's scores before a CBPE-style estimate. Our harm tier applies the weights directly to the model's correctness on a labeled reference set. We do not claim the estimator as new. Our question is whether its estimated drop, used as an alert with a frozen threshold, separates harmful from harmless shifts before delayed labels arrive.

**Delayed labels.** Learning and evaluation when labels arrive late or never are studied in [20], [21]. We use a fixed delay of $L$ batches and ask how early a monitor can warn relative to the moment delayed labels could confirm a drop.

**Pre-registration.** Fixing hypotheses and analysis before seeing results reduces analytic flexibility [22]; we apply it to an ML monitoring evaluation, including thresholds.

## III. Problem Setting and Protocol

A frozen binary classifier $f$ is trained on a labeled reference set $R$ and then scores a stream of batches $B_1, B_2, \ldots$ of 300 rows. The labels of batch $b$ become usable at batch $b + \max(L, 1)$, with label delay $L$ (primary setting $L = 5$). The first 8 batches after deployment form a burn-in period during which no alerts are raised.

**Degradation event.** The baseline is the mean accuracy of batches 0–4. An event begins at the first of two consecutive batches whose accuracy is at least 5 points below the baseline.

**Warning tiers.** For an intervention starting at batch $s$ and an event at batch $e$, a *predictive warning* (T1) is an alert in $[s, e)$, and a *pre-confirmation warning* (T2) is an alert in $[s, e + 1 + \max(L, 1))$, i.e., before delayed labels could confirm the event. Fig. 1 illustrates the tiers.

**False alerts.** Clean batches are the batches of unshifted streams and the pre-shift batches of every other stream. The budget is 5 false alerts per 100 clean batches. For shifts that cause no event (harmless shifts) we report the share of episodes with any alert after onset.

**Algorithm 1** Building one benchmark episode

**Require:** dataset $(X, y)$, kind, seed
draw a block start where both classes are $\geq$5% of the fit rows and the pool
$R \leftarrow$ next 2000 rows; fit $f$ on the first 1500, importance on the last 500
$P \leftarrow$ next 20,000 rows (pool); draw onset $s \in [10, 16]$
**if** kind is a shift **then**
    pick feature $j$ (important or unimportant half), sign $\sigma$
**end if**
**for** $t = 0, \ldots, 44$ **do**
    **if** kind is a shift and $t \geq s$ **then**
        sample 300 rows of $P$ with weights $\propto \exp(\lambda(t)\, \sigma\, z_j)$
    **else**
        sample 300 rows of $P$ uniformly
    **end if**
    **if** kind is a label change and $t \geq s$ **then**
        flip labels of rows with $z_k > 0$ w.p. $0.3 \min(1, (t - s + 1)/15)$
    **end if**
**end for**
measure accuracy of $f$ per batch; label the episode harmful, harmless or clean by the event rule

**Pre-registration.** For each experiment we committed, before obtaining its confirmation data, a configuration module defining datasets, seeds, the method, the threshold rule and each hypothesis with its pass rule. Thresholds were chosen on development data only and frozen with recorded SHA-256 fingerprints; the confirmation script refuses to run if they change. Deviations (for example, shuffling a dataset whose rows are stored sorted by class) were logged with date and reason before any affected episode was built.

## IV. Benchmark

Each episode draws a contiguous block of a dataset: 2000 reference rows (1500 to fit $f$, 500 held out to compute its permutation feature importance) and a pool of the next 20,000 rows. Because the reference and the pool come from one period, an unshifted stream has no built-in drift. The stream has 45 batches sampled with replacement from the pool; every row keeps its true label. From an onset batch $s \in [10, 16]$:

- *gradual / abrupt shift*: rows are sampled with weights $\propto \exp(\lambda(t)\, \sigma\, z_j)$, where $z_j$ is a standardized feature, $\sigma = \pm 1$, and $\lambda$ ramps to 1.5 over 15 batches (gradual) or jumps to 1.5 (abrupt). The feature is drawn from the more or the less important half with equal probability;
- *label change*: labels are flipped with a probability ramping to 0.3 for rows with $z_k > 0$;
- *clean*: uniform sampling.

Algorithm 1 summarizes the construction. Each dataset contributes 10 clean, 30 gradual, 15 abrupt and 10 label-change episodes.

A shift episode is *harmful* if its event starts after onset and *harmless* if no event occurs; episodes with an event before onset are excluded. Across all data groups only about one shift in five was harmful, and harm was barely related to the importance of the shifted feature; it did not change when the shift strength $\lambda$ was raised from 1.5 to 4.0. This failed a pre-registered v6 gate requiring at least 25% harmful shifts on development data (17% at $\lambda = 1.5$). Because no allowed

TABLE I
DATA GROUPS AND THEIR USE (65 EPISODES PER DATASET).

| Group | Datasets | Use |
|---|---|---|
| v6 development | 8 | design, thresholds, all later tuning |
| v6 held-out | 5 | one-time v6 verdict |
| v7 confirmation | 6 | one-time drift-watch verdict |
| v8 confirmation | 5 | one-time harm-tier verdict |
| v10 confirmation | 6 | one-time test of alerting rules |
| Natural streams | 13 | H5/C5 (no injected shift) |

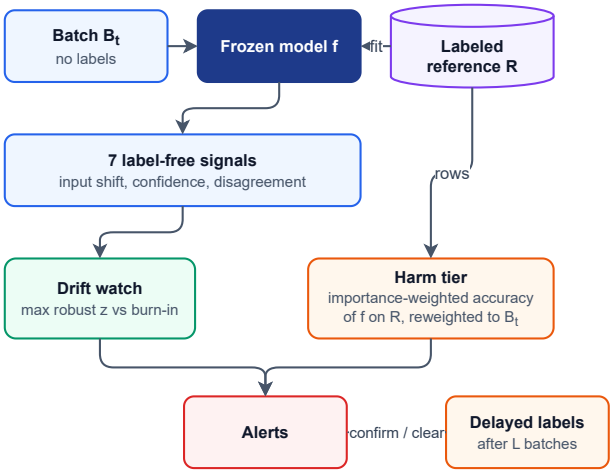



Fig. 2. Two-tier early-warning monitor. The drift watch flags changes in inputs or model behavior; the harm tier estimates whether accuracy is falling; delayed labels confirm or clear the warning.

strength passed, we proceeded at $\lambda = 1.5$ by a decision recorded before any held-out data were generated.

Datasets come from OpenML [23]. The first study (v6) used 13 datasets, 8 for development (covertype, CovPokElec, Hyperplane, KDDCup99, LED, poker-hand, run-or-walk, vehicle-SensIT) and 5 held out (airlines, bank-marketing, electricity, ldpa, SEA). Later confirmations used fresh datasets: adult, MiniBooNE, higgs, nomao, numerai28.6 and Click-prediction-small (v7); default-of-credit-card-clients, kick, house-16H, 2dplanes and mv (v8). The primary model is logistic regression [24]; gradient boosting was a robustness check in v6. All 13 v6 datasets had already been used as natural-order streams in the exploratory studies v1–v5, and the natural-stream windows of H5/C5 come from those studies, so the v6 held-out data were held out from v6 tuning only. The v7, v8 and v10 confirmation datasets were downloaded only after the corresponding design and thresholds were frozen, and had not been used before. Table I summarizes how the data were used.

## V. MONITORING METHODS

Fig. 2 summarizes the final two-tier design: both warning tiers use only label-free signals and the labeled reference set, and delayed labels later confirm or clear a warning.

**Signals.** Per batch the monitor computes, without labels: a fused drift score combining per-feature statistics (KS, PSI) with an embedding MMD and a confidence component; the mean KS statistic; a relevance-weighted shift (KS weighted by the model's permutation importance); disagreement between $f$ and two auxiliary models trained on the same reference rows; mean confidence and entropy; and the shift in the positive-prediction rate.

**v6: learned risk calibrator.** A logistic model maps these raw signals, their recent means and slopes, and the observed accuracy drop from matured labels to the probability that a degradation starts or continues within 5 batches. Thresholds are chosen on development data to maximize T1 within the false-alert budget.

**v7: drift watch (stream-normalized).** Each signal $x_t$ is converted to a robust z-score using the burn-in, $z_t = (x_t - \mathrm{med}(x_{0:8}))/\max(1.4826\,\mathrm{MAD}(x_{0:8}), 0.01)$, and the drift-watch score is the maximum z-score over seven label-free signals. A threshold of 4.0 deviations, chosen on development data, is comparable across streams that start in a stable period.

**v8: harm tier.** For each batch $B_t$, a logistic domain classifier $g_t$ is trained to separate reference rows (label 0) from batch rows (label 1). For reference row $x_i$ with correctness $c_i = \mathbb{1}[f(x_i) = y_i]$, the importance weight and the label-free accuracy estimate are

$$w_i = \min\left(\frac{\rho_i}{\bar{\rho}}, 20\right), \quad \rho_i = \frac{g_t(x_i)}{1 - g_t(x_i)}, \qquad \hat{a}_t = \frac{\sum_i w_i c_i}{\sum_i w_i}, \tag{1}$$

where $\bar{\rho}$ is the mean odds over the reference. Under covariate shift, $\hat{a}_t$ targets the accuracy of $f$ on $B_t$ [17], [18]. The harm score is $h_t = \mathrm{med}(\hat{a}_{0:8}) - \hat{a}_t$, the estimated drop in accuracy points since the burn-in. Its threshold (an estimated drop of 1.15 points) was chosen on development data to maximize T2 on harmful shifts minus the alert share on harmless shifts, within the budget.

## VI. RESULTS

All results use logistic regression and label delay $L = 5$. Intervals are 95% bootstrap intervals over episodes.

### A. v6: the original design fails

Table II shows the pre-registered verdict on the 5 held-out datasets (53 harmful and 169 harmless shifts). The learned calibrator warned before the drop on only 22% of harmful gradual shifts and did not beat waiting for delayed labels. The fused drift score's higher warning rates came from alerting almost constantly. Among the secondary settings (Table III), the gradient-boosting system warned before confirmation more often than the delayed-label trend when only 20% of rows were ever labeled, but its false alerts exceeded the budget at most delays, and the logistic system stayed below the label trend; we therefore make no claim from these settings.

### B. Why: thresholds in raw units do not transfer

Fig. 3 compares false-alert rates on development data with those on data never used for threshold selection. Scores in dataset-specific units broke the budget by a factor of 3.5–9 (fused drift score 4.4 → 36.9; mean KS 4.4 → 44.6),

TABLE II
V6 PRE-REGISTERED HYPOTHESES ON HELD-OUT DATASETS.

| Hypothesis (target) | Result | Verdict |
|---|---|---|
| H1: T1, harmful gradual ($\geq$0.60) | 0.22 [0.08, 0.36] | Fail |
| H2: T2 gain vs labels ($\geq$0.10) | $-$0.02 [$-$0.11, 0.08] | Fail |
| H3: harm AUROC ($\geq$0.80); harmless alerts ($\leq$0.10) | 0.59; 0.37 | Fail |
| H4: beats fused drift score ($\geq$4 of 5 datasets) | 2 of 5 | Fail |
| H5: natural streams, T2 gain ($\geq$0) | $-$0.01 [$-$0.08, 0.06] | Fail |

TABLE III
V6 SECONDARY SETTINGS WITH 20% OF LABELS (HELD-OUT): T2 ON HARMFUL SHIFTS, AND FALSE ALERTS PER 100 CLEAN BATCHES.

| Model, delay | Sys. T2 | Labels T2 | Sys. FA | Labels FA |
|---|---|---|---|---|
| Logistic, $L$=0 | 0.43 | 0.58 | 1.7 | 4.5 |
| Logistic, $L$=5 | 0.49 | 0.70 | 1.0 | 5.0 |
| Grad. boosting, $L$=0 | 0.64 | 0.48 | 3.9 | 4.0 |
| Grad. boosting, $L$=2 | 0.73 | 0.57 | 5.7 | 5.1 |
| Grad. boosting, $L$=5 | 0.73 | 0.61 | 5.3 | 4.9 |
| Grad. boosting, $L$=10 | 0.82 | 0.80 | 7.7 | 5.9 |

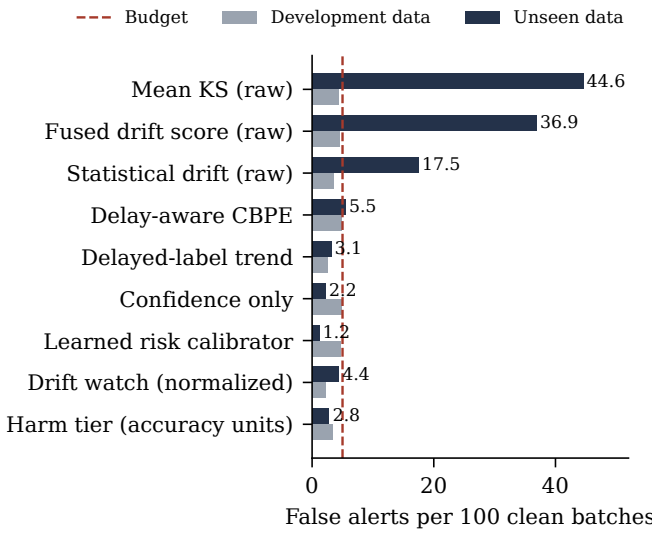



Fig. 3. False alerts per 100 clean batches with thresholds chosen on development data, evaluated on development and on unseen datasets (v6 held-out data for the v6 methods; v7 and v8 confirmation data for the drift watch and the harm tier). Dashed line: budget of 5.

while the learned calibrator under-alerted (4.6 $\rightarrow$ 1.2). Scores expressed relative to the stream's own normal (drift watch, 2.3 $\rightarrow$ 4.4) or in accuracy points (harm tier, 3.4 $\rightarrow$ 2.8) stayed within the budget on their confirmation data. It did not hold everywhere, though. The drift watch broke the budget on natural streams, and the harm rule on the v10 datasets (Section VI-F). An ablation confirmed the mechanism inside the calibrator: removing the raw disagreement levels, which differ widely across datasets, raised its T1 from 0.22 to 0.42.

### C. v7: the drift watch tier

On six fresh datasets (39 harmful gradual shifts) the drift watch warned before the accuracy drop on 0.87 [0.77, 0.97] of harmful gradual shifts, a median of 6 batches ahead, with 4.4 false alerts per 100 clean batches, and beat the fused drift score on 5 of 6 datasets (Table IV). Its pre-confirmation warning rate (1.00 vs 0.96 for the delayed-label trend) did not reach the pre-registered gain of 0.10. It also alerted on 96% of harmless shifts, so it detects drift but cannot tell whether the drift is harmful. On natural streams it failed (T2 0.44 vs 0.87 for the label trend) and exceeded the budget (8.0 false alerts per 100 clean batches, against 6.3). On these streams the median degradation began at batch 8, right at the end of the burn-in, so the burn-in was not a normal period. v7 also pre-registered a second tier, a logistic calibrator over the normalized signals; it failed its separation hypothesis (AUROC 0.59, alerts on 53% of harmless shifts), which motivated the reference-based harm tier of v8. The v7 verdict was a partial success.

### D. v8: the harm tier

On five further fresh datasets (64 harmful and 160 harmless shifts), the harm tier cut alarms on harmless shifts from 0.97 (drift watch) to 0.34 while still warning before labels could

TABLE IV
CONFIRMATION ON FRESH DATASETS (ONE-TIME RUNS).

| | v7 drift watch | v8 harm tier |
|---|---|---|
| Datasets | 6 | 5 |
| T1, harmful gradual shifts | 0.87 | 0.76 |
| T2, harmful shifts | 1.00 | 0.83 |
| False alerts / 100 clean batches | 4.4 | 2.8 |
| Harmless shifts with an alert | 0.96 | 0.34 |
| Harm AUROC | 0.51 | 0.78 |
| Pre-registered verdict | partial success | partial success |

confirm on 0.83 of harmful shifts and before the drop on 0.76 of harmful gradual shifts, at 2.8 false alerts per 100 clean batches. It separated harmful from harmless shifts with an AUROC of 0.78 [0.70, 0.86], against 0.59 for the drift watch (gain 0.20 [0.09, 0.30]). It passed two of four hypotheses and missed the AUROC target of 0.80 and the harmless-alarm target of 0.10. Per dataset (Table V), the harm tier separated harm well on three of the four datasets that produced harmful shifts and was near chance on kick. Post hoc, we scored other signals on the same data in the same way (Table VI). Confidence drop, entropy and delay-aware CBPE reached only 0.56–0.59, each clearly below the harm tier. kNN reference accuracy and disagreement were not distinguishable from it; both also learn from the labeled reference. Confidence drop had reached 0.77 on the v7 data (Table VIII), so confidence alone was not stable across datasets. Fig. 4 shows one harmful gradual shift: the harm tier first alerts at batch 16, 14 batches before the accuracy drop and 20 before delayed labels confirm it.

TABLE V
HARMFUL-VS-HARMLESS AUROC PER V8 CONFIRMATION DATASET.

| Dataset | Harmful | Harmless | Harm tier | Drift watch |
|---|---|---|---|---|
| 2dplanes | 14 | 31 | 0.98 | 0.64 |
| credit default | 8 | 36 | 0.89 | 0.58 |
| house-16H | 26 | 19 | 0.78 | 0.60 |
| kick | 16 | 29 | 0.61 | 0.60 |
| mv | 0 | 45 | – | – |

TABLE VI
HARM AUROC OF OTHER SIGNALS ON THE V8 CONFIRMATION DATA (POST HOC), SCORED LIKE THE HARM TIER. LAST COLUMN: 95% PAIRED BOOTSTRAP INTERVAL OF THE HARM TIER'S AUROC MINUS THE METHOD'S.

| Method | Ref. labels | AUROC | Harm tier minus |
|---|---|---|---|
| Harm tier (frozen) | yes | 0.78 | – |
| kNN reference accuracy | yes | 0.76 | [−0.02, 0.06] |
| Disagreement | yes | 0.73 | [−0.02, 0.14] |
| Entropy | no | 0.59 | [0.11, 0.28] |
| Drift watch (frozen) | no | 0.59 | [0.09, 0.30] |
| Confidence drop | no | 0.59 | [0.12, 0.29] |
| Delay-aware CBPE | no | 0.56 | [0.12, 0.33] |

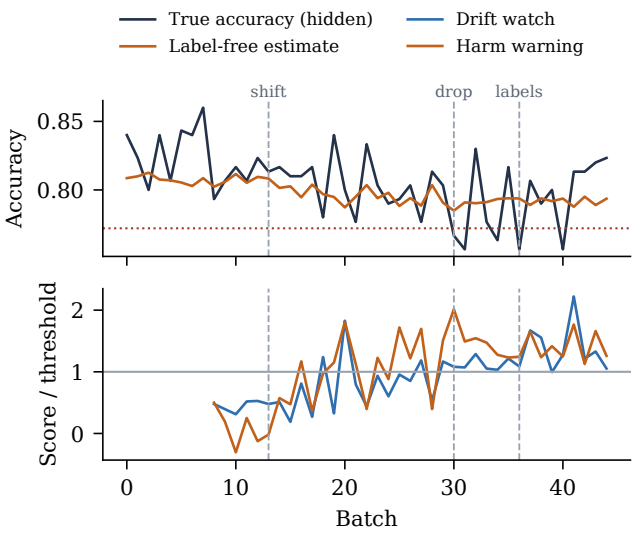



Fig. 4. A harmful gradual shift on an unseen dataset (credit default, $L = 5$). Top: true accuracy (hidden from the monitor) and the label-free estimate; dotted line: 5 points below baseline. Bottom: tier scores divided by their thresholds; above 1 is an alert.

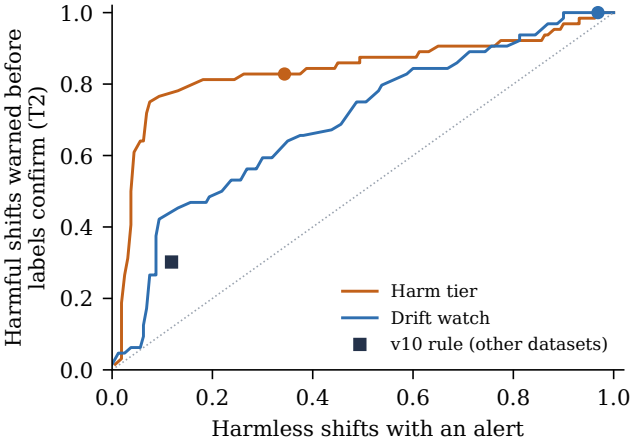



Fig. 5. Post hoc operating curves on the v8 confirmation data: each point is a threshold. Circles: frozen thresholds. Square: the stricter v10 rule, frozen on development data and evaluated on six different datasets.

### *E. Why harmless shifts still alert*

In a post-hoc analysis of the v8 confirmation data, streams with no shift at all raised a harm alert in 40% of episodes and harmless shifts in 34%, against 88% for harmful shifts. The harm tier therefore treats harmless shifts like no shift; the remaining alarms are background false alerts that accumulate over about 33 monitored batches. With a per-batch rate of 2.8 per 100, the chance of at least one alert in 33 batches is about 0.6. Our 10% target did not match our own per-batch budget. We noticed this only after the run and left the verdict unchanged.

Fig. 5 shows the full trade-off on the same data, post hoc. Over most of the range of harmless alarms the harm tier warns on far more harmful shifts than the drift watch. On this data a stricter harm threshold would appear better than the frozen one, but when that idea was frozen and tested on six other datasets (v10, square marker) it warned on only 30% of harmful shifts. Picking a threshold from this curve after seeing the results would have misled us.

### *F. Pre-registered follow-ups that failed*

Two further experiments tried to improve the harm tier (Table VII). In v9, a gradient-boosting domain classifier and an error predictor trained on out-of-fold reference correctness both separated harm worse than the linear estimator on development data (AUROC 0.70 vs 0.77), In a post hoc check, ensembles of these estimators, kNN accuracy and confidence (averaging within-dataset ranks) reached at most 0.76. Scored the same way, the linear estimator alone reached 0.75. The 0.80 development gate failed, so no fresh data were used. In v10, stricter thresholds, smoothing, CUSUM, stream-normalized scores and gating by the drift watch all reduced harmless alarms at the cost of early warnings. The best rule passed the development gate but, on six fresh datasets, warned before labels on only 0.30 of harmful shifts against a required 0.50, while reducing harmless alarms to 0.12. Neither was adopted. On the same six datasets the frozen v8 rule warned before labels on 0.74 of harmful shifts but alerted on 58% of harmless shifts and raised 12.7 false alerts per 100 clean batches, more than twice the budget, so its calibration did not hold on every new dataset.

## VII. DISCUSSION AND LIMITATIONS

**Comparable units made thresholds transfer.** The step from v6 to v7 did not need a better model, since the drift watch is simpler than the v6 calibrator; what changed was the

TABLE VII
FOLLOW-UP EXPERIMENTS (NOT ADOPTED).

| Attempt | Outcome |
|---|---|
| v9: nonlinear harm estimators, ensembles | Development AUROC 0.70 (vs 0.77); ensembles $\leq$0.76 (post hoc); gate failed |
| v8 rule on the v10 datasets | Harmless 0.58, false alerts 12.7 per 100 (>5) |
| v10: stricter alerting rules | Fresh data: harmless 0.12 but T2 0.30 (<0.50); confirmation failed |

TABLE VIII
EACH NORMALIZED SIGNAL USED ALONE (V7 CONFIRMATION DATA, POST HOC).

| Family | Signal | T1 | Harmless | AUROC |
|---|---|---|---|---|
| Input shift | fused drift score | 0.90 | 0.95 | 0.52 |
| | mean KS | 0.90 | 0.95 | 0.53 |
| | relevance-weighted shift | 0.79 | 0.92 | 0.51 |
| | positive-rate shift | 0.79 | 0.73 | 0.54 |
| Model behavior | disagreement | 0.49 | 0.42 | 0.72 |
| | confidence drop | 0.46 | 0.24 | 0.77 |
| | entropy | 0.41 | 0.25 | 0.75 |

scale of the signals. It did not solve everything: the harm rule still broke the budget on the v10 datasets, and the drift watch did on natural streams.

**Drift and harm need different signals.** Using each normalized signal alone as the drift watch on the v7 confirmation data (Table VIII), input-shift signals warned early on 79–90% of harmful gradual shifts but could not tell harm from harmlessness (AUROC about 0.5), whereas model-behavior signals were harm-aware (AUROC 0.72–0.77) but warned later.

On the v8 data, the methods that learn from the labeled reference separated harmful shifts best (Table VI), and none of our attempts exceeded about 0.78 without labels.

**Lessons for practitioners.** Validate alert thresholds on data from a different source than the one used to choose them. Keep a labeled reference sample, since in our data only methods that used it separated harm consistently across datasets.

**Threats to validity.** The v7 and v8 designs were motivated by the v6 failure, so they are informed by data we had already seen; we mitigated this by confirming each on datasets downloaded only after its design was frozen, but the sequence of experiments was not planned in advance. The v6 datasets had been explored in v1–v5, so the v6 held-out results are not fully out of sample. Harmful-shift counts per confirmation group (52–64; 39 gradual in v7) give confidence intervals of roughly $\pm$0.1. Our event rule (two batches at least 5 points below baseline) and horizon are conventional choices that we did not vary.

**Limitations.** Shifts are constructed by re-sampling real rows, which preserves $P(y \mid x)$ and thus favors covariate-shift methods; changes in the label relationship are invisible to every label-free signal until labels arrive. The main model is logistic regression, the burn-in assumes a stable start (which failed on natural streams), and the per-dataset numbers of harmful shifts are small, so intervals are wide. We did not compare the harm tier with PAPE [19], which may estimate accuracy more precisely, and we did not control false alarms over time as sequential tests do [12].

## VIII. CONCLUSION

In pre-registered, one-time tests on unseen data, the stream-normalized drift watch gave early warning within its false-alert budget on benchmark streams, but not on natural streams. The harm tier cut harmless-shift alarms from 97% to 34% (AUROC 0.78), yet broke the budget on a later set of datasets. Without labels for the monitored batches, telling harmful from harmless drift worked only partly, and only with a labeled reference set.

## REPRODUCIBILITY

Code, configurations, frozen thresholds, verdict scripts and a one-command reproduction script are available at https://github.com/rushikbuilds/ML-Drift-Monitoring-System. In a clean environment, ten regenerated v8 episodes matched the saved ones to within $4 \times 10^{-11}$, and the v6, v7 and v8 verdicts reproduce exactly. The v9 and v10 code is on separate branches of the repository, and the post hoc analyses are in `scripts/run_posthoc.py`.


## ACKNOWLEDGMENT

The authors thank the Department of Computer Engineering, COEP Technological University, Pune, for its support. We used an AI assistant (Claude, Anthropic) extensively in this work, as IEEE and arXiv policies require us to state: to help write and debug the experiment code, to make the figures from saved results, and to draft the text of all sections, which we then revised. The project's code produced every result, we checked them, and we take full responsibility for the paper.


## REFERENCES


[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[2] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *IEEE International Conference on Big Data*, 2017.

[3] S. Rabanser, S. Günnemann, and Z. C. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[4] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[5] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[6] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence – SBIA 2004*, ser. Lecture Notes in Computer Science, vol. 3171. Springer, 2004, pp. 286–295.

[7] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proceedings of the SIAM International Conference on Data Mining (SDM)*, 2007.

[8] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[9] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, "A kernel two-sample test," *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.

[10] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, Eds., *Dataset Shift in Machine Learning*. MIT Press, 2009.
[11] Z. C. Lipton, Y.-X. Wang, and A. J. Smola, "Detecting and correcting for label shift with black box predictors," in *International Conference on Machine Learning (ICML)*, 2018.
[12] A. Podkopaev and A. Ramdas, "Tracking the risk of a deployed model and detecting harmful distribution shifts," in *International Conference on Learning Representations (ICLR)*, 2022.
[13] NannyML, "Performance estimation: Confidence-based performance estimation (CBPE)," Documentation, https://nannyml.readthedocs.io/en/v0.10.3/how_it_works/performance_estimation.html, 2023.
[14] S. Garg, S. Balakrishnan, Z. C. Lipton, B. Neyshabur, and H. Sedghi, "Leveraging unlabeled data to predict out-of-distribution performance," in *International Conference on Learning Representations (ICLR)*, 2022.
[15] D. Guillory, V. Shankar, S. Ebrahimi, T. Darrell, and L. Schmidt, "Predicting with confidence on unseen distributions," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
[16] Y. Jiang, V. Nagarajan, C. Baek, and J. Z. Kolter, "Assessing generalization of SGD via disagreement," in *International Conference on Learning Representations (ICLR)*, 2022.
[17] H. Shimodaira, "Improving predictive inference under covariate shift by weighting the log-likelihood function," *Journal of Statistical Planning and Inference*, vol. 90, no. 2, pp. 227–244, 2000.
[18] M. Sugiyama, M. Krauledat, and K.-R. Müller, "Covariate shift adaptation by importance weighted cross validation," *Journal of Machine Learning Research*, vol. 8, pp. 985–1005, 2007.
[19] J. Białek, J. Kivimäki, W. Kuberski, and N. Perrakis, "Estimating model performance under covariate shift without labels," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
[20] M. Grzenda, H. M. Gomes, and A. Bifet, "Delayed labelling evaluation for data streams," *Data Mining and Knowledge Discovery*, vol. 34, pp. 1237–1266, 2020.
[21] M. Umer and R. Polikar, "Comparative analysis of extreme verification latency learning algorithms," arXiv:2011.14917, 2020.
[22] E. van Miltenburg, C. van der Lee, and E. Krahmer, "Preregistering NLP research," in *Proceedings of NAACL-HLT*, 2021.
[23] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *SIGKDD Explorations*, vol. 15, no. 2, pp. 49–60, 2014.
[24] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.